

1. Maximum Product of Two Elements in an Array

1464. Maximum Product of Two Elements in an Array

Solved 

Easy

Topics

Companies

Hint

Given the array of integers `nums`, you will choose two different indices `i` and `j` of that array. Return the maximum value of $(\text{nums}[i]-1) * (\text{nums}[j]-1)$.

Example 1:

Input: `nums = [3,4,5,2]`

Output: 12

Explanation: If you choose the indices `i=1` and `j=2` (indexed from 0), you will get the maximum value, that is, $(\text{nums}[1]-1) * (\text{nums}[2]-1) = (4-1) * (5-1) = 3 * 4 = 12$.

Code:

```
1 class Solution {
2     public int maxProduct(int[] nums) {
3         int max = 0;
4         int smax = 0;
5         for(int i=0; i<nums.length; i++){
6             if(nums[i] > max){
7                 smax = max;
8                 max = nums[i];
9             }
10            else if(nums[i] > smax){
11                smax = nums[i];
12            }
13        }
14        return (max-1)*(smax-1);
15    }
16 }
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

2. Second Largest Element

Code:

```
1  import java.util.Scanner;
2  public class Array_SecondLargest {
    Run | Debug
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          int num = sc.nextInt();
6          int arr[] = new int[num];
7          for(int i=0; i<num; i++){
8              arr[i] = sc.nextInt();
9          }
10         int max = Integer.MIN_VALUE;
11         int secondMax = Integer.MIN_VALUE;
12         for(int i=0; i<num; i++){
13             if(arr[i] > max){
14                 secondMax = max;
15                 max = arr[i];
16             }
17             else if(arr[i] > secondMax && arr[i] != max){
18                 secondMax = arr[i];
19             }
20         }
21         System.out.println(secondMax);
22     }
23 }
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

3. Find Peak Element.

162. Find Peak Element

Solved 

Medium

 Topics

 Companies

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `2`

Explanation: 3 is a peak element and your function should return the index number 2.

Code:

```
1 class Solution {
2     public int findPeakElement(int[] nums) {
3         int len = nums.length;
4         int max = nums[0];
5         int index = 0;
6         for (int i = 0; i < len; i++) {
7             if (nums[i] > max) {
8                 max = nums[i];
9                 index = i;
10            }
11        }
12        return index;
13    }
14 }
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

4. Search in Rotated Array

33. Search in Rotated Sorted Array

Solved

Medium Topics Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return the *index of target* if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: `4`

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: `-1`

Code:

```
1 class Solution {
2     public int search(int[] nums, int target) {
3         int low = 0;
4         int high = nums.length - 1;
5         while(low <= high){
6             int mid = (low + high) / 2;
7             if(nums[mid] == target){
8                 return mid;
9             }
10            if(nums[low] <= nums[mid]){
11                if(target >= nums[low] && target <= nums[mid]){
12                    high = mid - 1;
13                }
14                else{
15                    low = mid + 1;
16                }
17            }
18            else{
19                if(target > nums[mid] && target <= nums[high]){
20                    low = mid + 1;
21                }
22                else {
23                    high = mid - 1 ;
24                }
25            }
26        }
27        return -1;
28    }
29 }
```

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

5. Fruits in the Basket II

3477. Fruits Into Baskets II

Easy Topics Companies Hint

You are given two arrays of integers, `fruits` and `baskets`, each of length `n`, where `fruits[i]` represents the **quantity** of the i^{th} type of fruit, and `baskets[j]` represents the **capacity** of the j^{th} basket.

From left to right, place the fruits according to these rules:

- Each fruit type must be placed in the **leftmost available basket** with a capacity **greater than or equal** to the quantity of that fruit type.
- Each basket can hold **only one** type of fruit.
- If a fruit type **cannot be placed** in any basket, it remains **unplaced**.

Return the number of fruit types that remain unplaced after all possible allocations are made.

Code:

```
</> Code
Java ▾ 🔒 Auto

1  class Solution {
2      public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
3          int count = 0;
4          for (int i = 0; i < fruits.length; i++) {
5              boolean placed = false;
6              for (int j = 0; j < baskets.length; j++) {
7                  if (baskets[j] >= fruits[i]) {
8                      baskets[j] = -1;
9                      placed = true;
10                     break;
11                 }
12             }
13             if (!placed) {
14                 count++;
15             }
16         }
17         return count;
18     }
19 }
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

6. Best Time to Buy and Sell Stock

121. Best Time to Buy and Sell Stock

Solved 

Easy

Topics

Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: `5`

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = $6 - 1 = 5$.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Code:

```
1 class Solution {
2     public int maxProfit(int[] prices) {
3         int ans = 0, min = prices[0];
4         for(int v : prices){
5             ans = Math.max(ans, v-min);
6             min = Math.min(min, v);
7         }
8         return ans;
9     }
10 }
11
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

7. Two Sum

1. Two Sum

Solved ✓

Easy

Topics

Companies

Hint

You are given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`

Output: `[1,2]`

Code:

```
1 class Solution {
2     public int[] twoSum(int[] nums, int target){
3         int result[] = new int[2];
4         for(int i=0; i<nums.length; i++){
5             for(int j=i+1; j<nums.length; j++){
6                 if(nums[i] + nums[j] == target){
7                     result[0] = i;
8                     result[1] = j;
9                 }
10            }
11        }
12        return result;
13    }
14 }
```

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

8. How Many Numbers Are Smaller Than the Current Number

1365. How Many Numbers Are Smaller Than the Current Number

Solved 

Easy

Topics

Companies

Hint

Given the array `nums`, for each `nums[i]` find out how many numbers in the array are smaller than it. That is, for each `nums[i]` you have to count the number of valid `j`'s such that `j != i` and `nums[j] < nums[i]`.

Return the answer in an array.

Example 1:

Input: `nums = [8,1,2,2,3]`

Output: `[4,0,1,1,3]`

Explanation:

For `nums[0]=8` there exist four smaller numbers than it (1, 2, 2 and 3).

For `nums[1]=1` does not exist any smaller number than it.

For `nums[2]=2` there exist one smaller number than it (1).

For `nums[3]=2` there exist one smaller number than it (1).

For `nums[4]=3` there exist three smaller numbers than it (1, 2 and 2).

Code:

```
1 class Solution {
2     public int[] smallerNumbersThanCurrent(int[] nums) {
3         int[] ans=new int[nums.length];
4         for (int i = 0; i < nums.length; i++) {
5             int c=0;
6             for (int j = 0; j < nums.length; j++) {
7                 if (nums[j] < nums[i]) {
8                     c++;
9                 }
10            }
11            ans[i] = c;
12        }
13        return ans;
14    }
15 }
```

Time Complexity: $O(n^2)$

Space Complexity: $O(n)$

9. Maximum Product of Three Numbers

628. Maximum Product of Three Numbers

Solved 

Easy

 Topics

 Companies

Given an integer array `nums`, find three numbers whose product is maximum and return the maximum product.

Example 1:

Input: `nums = [1,2,3]`

Output: 6

Example 2:

Input: `nums = [1,2,3,4]`

Output: 24

Code:

```
1  class Solution {
2      public int maximumProduct(int[] nums) {
3          Arrays.sort(nums);
4          int n = nums.length;
5          return Math.max(
6              nums[0] * nums[1] * nums[n - 1],
7              nums[n - 3] * nums[n - 2] * nums[n - 1]
8          );
9      }
10 }
```

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$

10. Remove Duplicates from Sorted Array

26. Remove Duplicates from Sorted Array

Solved ✓

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Code:

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int n = nums.length;
4         int count = 1;
5         if (n==0)
6             return 0;
7         for ( int i=1; i<n; i++){
8             if(nums[i] != nums[i-1]){
9                 nums[count] = nums[i];
10                count++;
11            }
12        }
13        return count;
14    }
15 }
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

NAME: AKARSH SRIVASTAVA

UNIVERSITY ROLL NO: 12515000131

CLASS ROLL NO: 06

SECTION: 2FB